



TECNOLÓGICO NACIONAL DE MÉXICO
Instituto Tecnológico de Apizaco

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN Y COMUNICACIONES

MATERIA:

Arquitectura en el desarrollo de software

DOCENTE:

José Juan Hernández Mora

Título:

arqui

PRESENTAN:

Índice de contenido

Capítulo 1 Tres propuestas de arquitectura	1
Arquitectura 1: Monolito en capas + MVC + documentación 4+1	1
Arquitectura 2: Monolito modular + Clean Architecture + 4+1	3
Arquitectura 3: Microservicios + API Gateway + 4+1.....	5
Capítulo 2 Comparación y selección de la mejor arquitectura	7
Capítulo 3 Diseño detallado de la arquitectura seleccionada	8

Propuesta de diseño de arquitectura para la aplicación web de recetas de Tlaxcala

Suposiciones de trabajo

Asumiendo que:

- La aplicación será **web responsive**.
- La persistencia será en una **base de datos NoSQL orientada a documentos JSON**.
- Habrá 3 roles:
 - **Visitante**
 - **Usuario registrado**
 - **Administrador**
- El volumen esperado es **pequeño a mediano**, con crecimiento futuro.
- La prioridad del proyecto es:
 1. cumplir requisitos,
 2. mantener bajo costo,
 3. facilitar mantenimiento,
 4. dejar abierta una evolución futura.

Capítulo 1 Tres propuestas de arquitectura

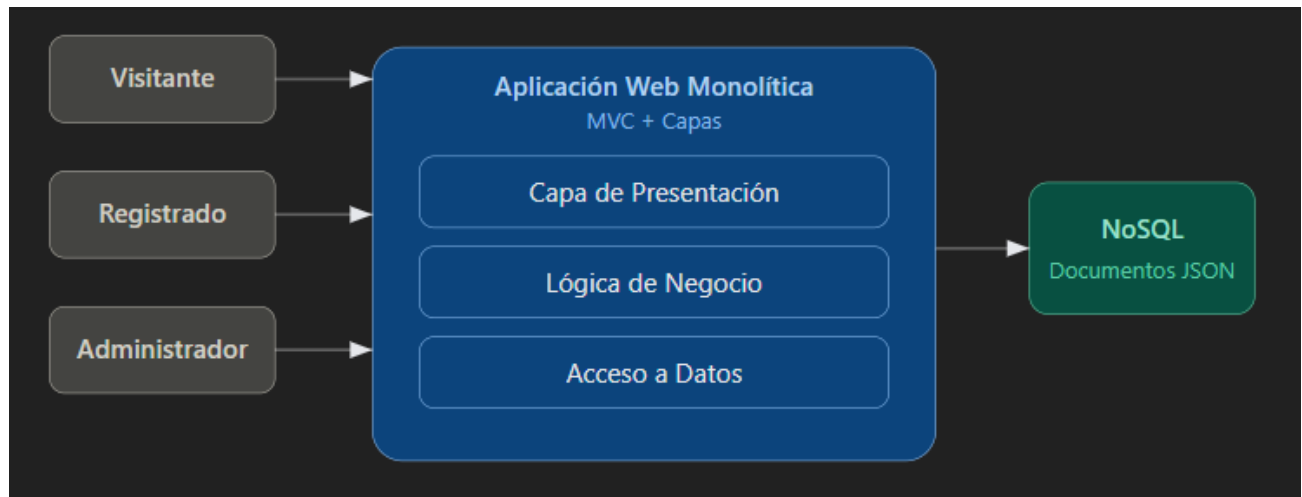
Arquitectura 1: Monolito en capas + MVC + documentación 4+1

Esta opción concentra toda la solución en una sola aplicación web, organizada por capas tradicionales:

- Presentación
- Lógica de negocio
- Acceso a datos
- Base de datos NoSQL

Se documenta con el modelo **4+1 vistas**, pero la implementación interna sigue un enfoque **MVC/por capas**.

Primero, un diagrama UML-style simplificado de esta alternativa:



Descripción

Ventajas

- Muy simple de construir.
- Menor costo inicial.
- Curva de aprendizaje baja.
- Adecuada para equipos pequeños.

Desventajas

- La lógica puede mezclarse con facilidad.
- Se vuelve más difícil de mantener cuando el sistema crece.
- Menor flexibilidad para escalar módulos concretos.
- Riesgo de mayor deuda técnica.

Cuando conviene

- Proyectos escolares pequeños o prototipos.
- Cuando el tiempo de entrega es muy corto.

Arquitectura 2: Monolito modular + Clean Architecture + 4+1

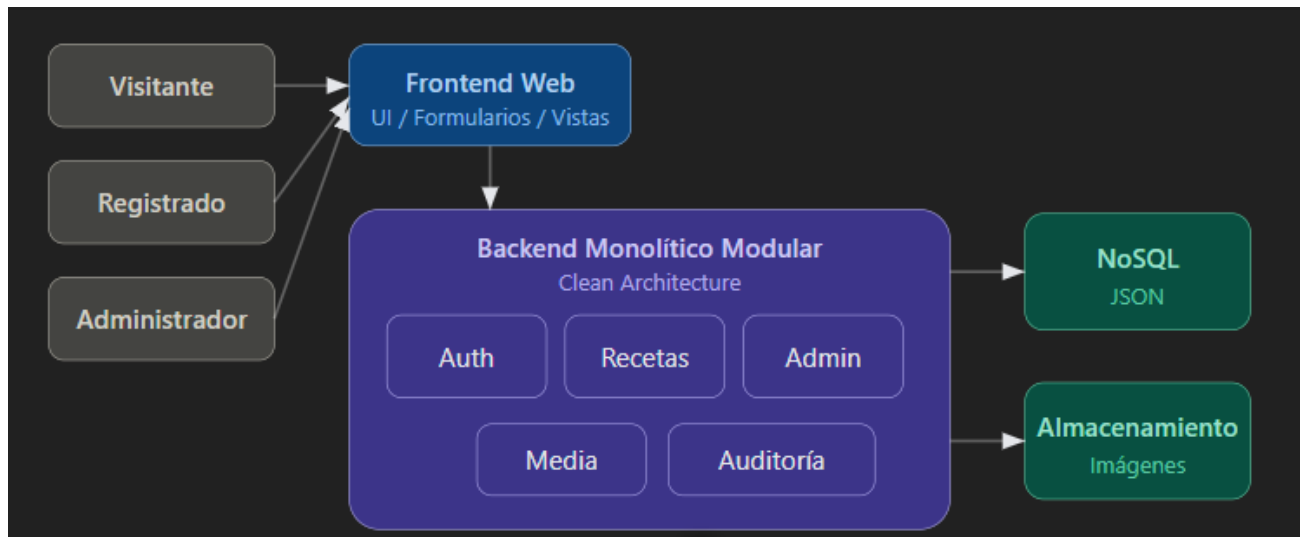
Esta alternativa separa claramente la solución en módulos de negocio, pero sigue desplegándose como una sola aplicación.

Internamente aplica **Clean Architecture** para desacoplar:

- dominio,
- casos de uso,
- infraestructura,
- interfaces.

Además, puede documentarse perfectamente con **4+1 vistas**.

Esta es, desde mi punto de vista, la **mejor candidata** para tu proyecto.



Descripción

Ventajas

- Buena separación de responsabilidades.
- Más mantenible que un monolito clásico.
- Menor complejidad que microservicios.
- Facilita pruebas unitarias, integración y refactorización.
- Permite crecer sin rehacer todo el sistema.

Desventajas

- Requiere más disciplina de diseño.
- Puede tardar un poco más al inicio que una solución MVC simple.
- Si no se respeta la arquitectura, se puede degradar a un monolito desordenado.

Cuando conviene

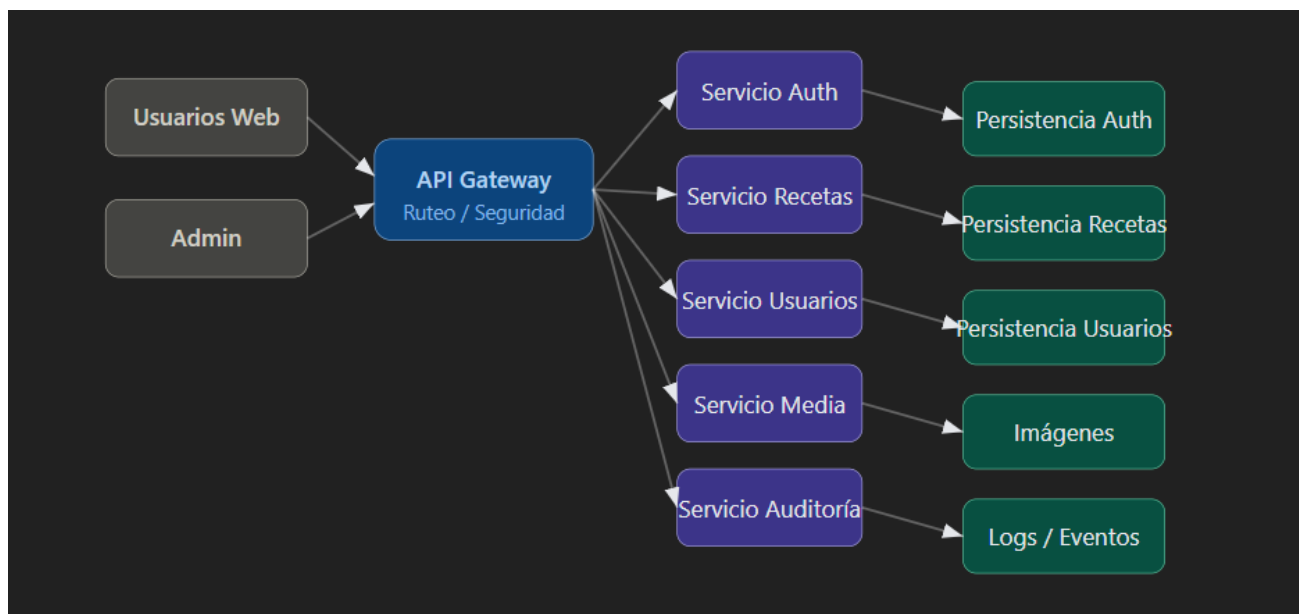
- Proyectos académicos serios.
- Sistemas institucionales con crecimiento moderado.
- Equipos pequeños o medianos que necesitan orden y mantenibilidad.

Arquitectura 3: Microservicios + API Gateway + 4+1

Esta alternativa divide el sistema en servicios independientes, por ejemplo:

- autenticación,
- gestión de recetas,
- gestión de usuarios,
- almacenamiento de imágenes,
- auditoría.

Cada servicio puede tener su propia base o persistencia asociada.



Descripción

Ventajas

- Escalabilidad alta.
- Despliegue independiente por servicio.
- Alta flexibilidad tecnológica.
- Muy útil para ecosistemas grandes.

Desventajas

- Mayor costo.
- Más complejidad operativa.
- Más esfuerzo en despliegue, monitoreo y seguridad.
- Sobredimensionada para este caso de uso.

Cuando conviene

- Plataformas grandes.
- Equipos maduros con DevOps.
- Requerimientos de alta escala o integración masiva.

Capítulo 2 Comparación y selección de la mejor arquitectura

Tabla comparativa

Criterio	Arquitectura 1: Monolito MVC	Arquitectura 2: Monolito modular + Clean	Arquitectura 3: Microservicios
Complejidad técnica	Baja	Media	Alta
Costo inicial	Bajo	Medio	Alto
Tiempo de implementación	Corto	Medio	Largo
Mantenibilidad	Media-baja	Alta	Alta
Escalabilidad	Media	Media-alta	Muy alta
Facilidad de pruebas	Media	Alta	Media
Riesgo de deuda técnica	Alto	Medio-bajo	Medio
Ajuste al proyecto	Medio	Muy alto	Bajo

Arquitectura seleccionada

La **Arquitectura 2: Monolito modular + Clean Architecture + 4+1** es la mejor alternativa.

Justificación

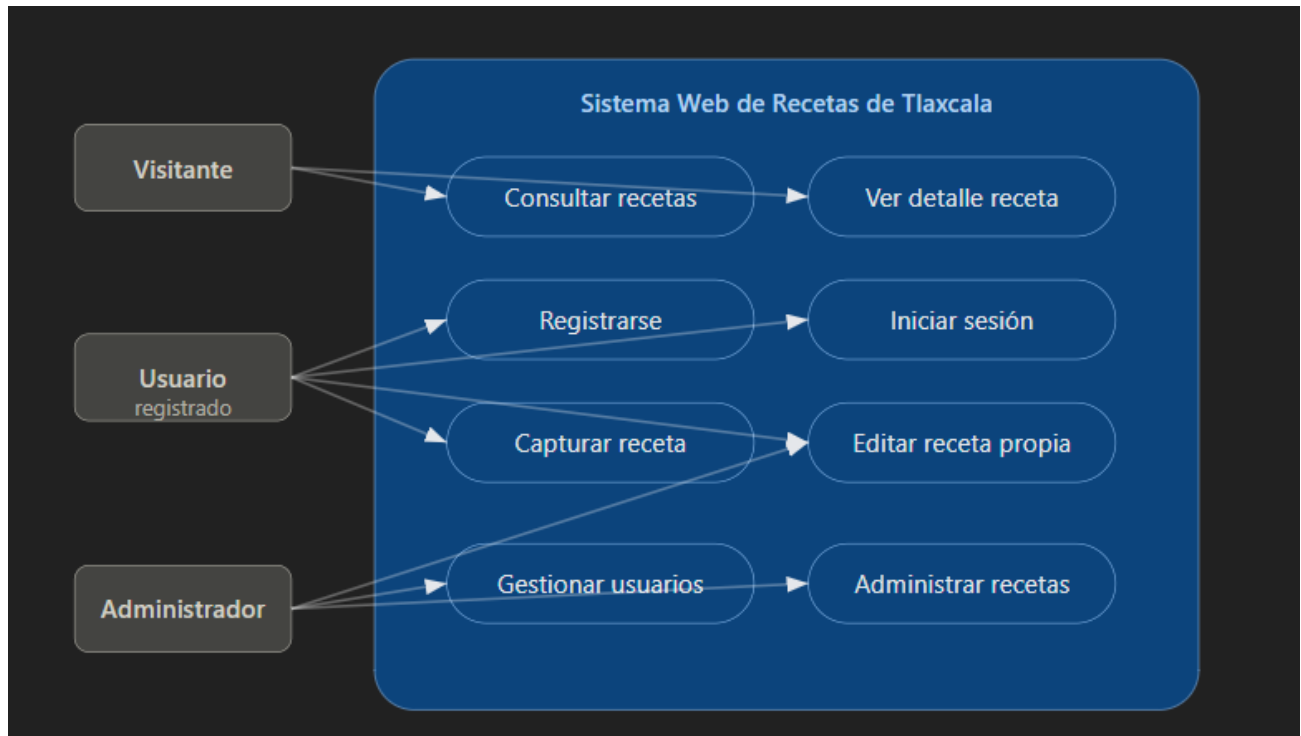
Se selecciona porque:

- cumple completamente con los requisitos funcionales;
- aprovecha la base NoSQL en JSON de forma natural;
- mantiene el sistema ordenado por módulos;
- permite que el equipo trabaje por responsabilidades;
- reduce deuda técnica frente a un monolito tradicional;
- evita la complejidad innecesaria de microservicios.

Capítulo 3 Diseño detallado de la arquitectura seleccionada

Visión general 4+1 de la arquitectura elegida

3.1 Vista de escenarios (+1)



Representa los casos de uso principales del sistema.

Actores

- Visitante
- Usuario registrado
- Administrador

Casos de uso principales

- Consultar recetas
- Ver detalle de receta
- Registrarse
- Iniciar sesión
- Capturar receta
- Editar receta propia
- Administrar recetas
- Administrar usuarios

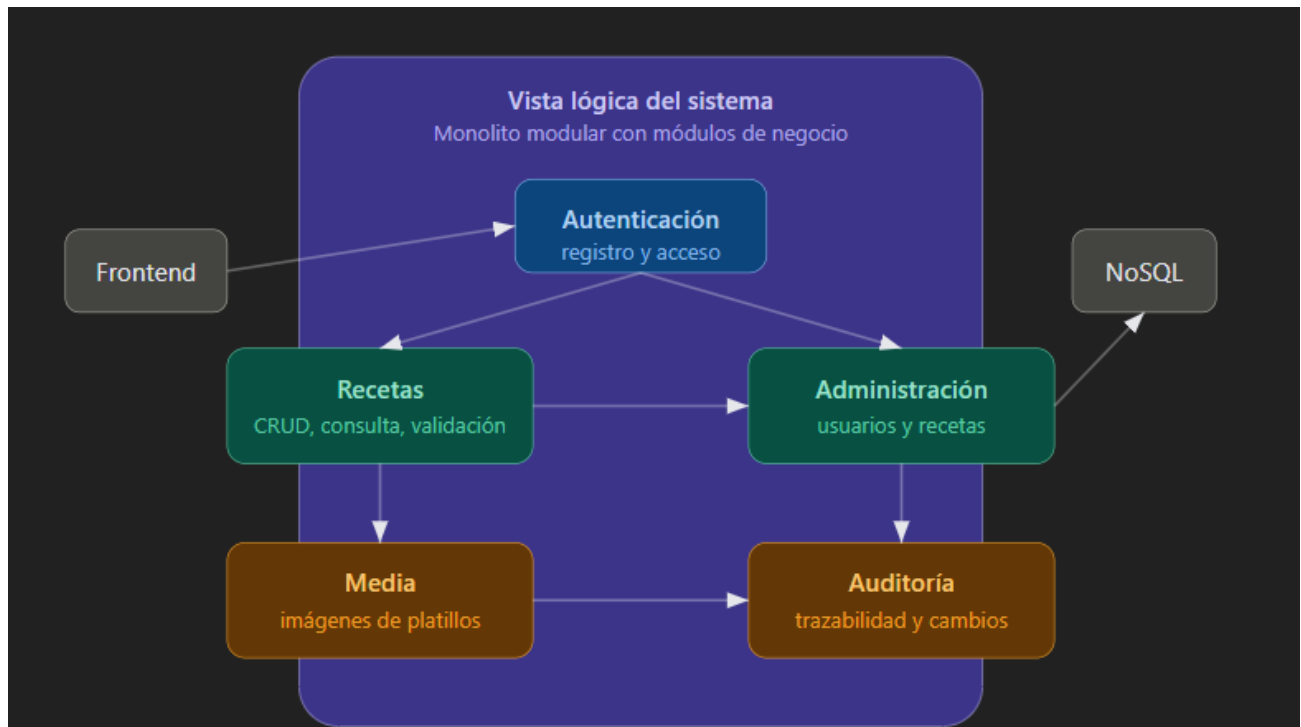
Texto descriptivo

La vista de escenarios demuestra que el sistema gira alrededor de la difusión gastronómica y la gestión controlada del contenido.

El visitante tiene acceso abierto a consulta; el usuario registrado colabora con contenido; y el administrador ejerce control total sobre información y usuarios.

3.2 Vista lógica

Describe los módulos funcionales y sus relaciones.



Módulos propuestos

- **Módulo de autenticación**
 - registro
 - inicio de sesión
 - control de acceso por rol
- **Módulo de recetas**
 - consulta pública
 - alta de recetas
 - edición de recetas propias
 - validación de las 6 secciones obligatorias
- **Módulo de administración**
 - gestión total de recetas
 - gestión de usuarios

- **Módulo de imágenes**
 - carga y asociación de imágenes a recetas
- **Módulo de auditoría**
 - registro de cambios relevantes

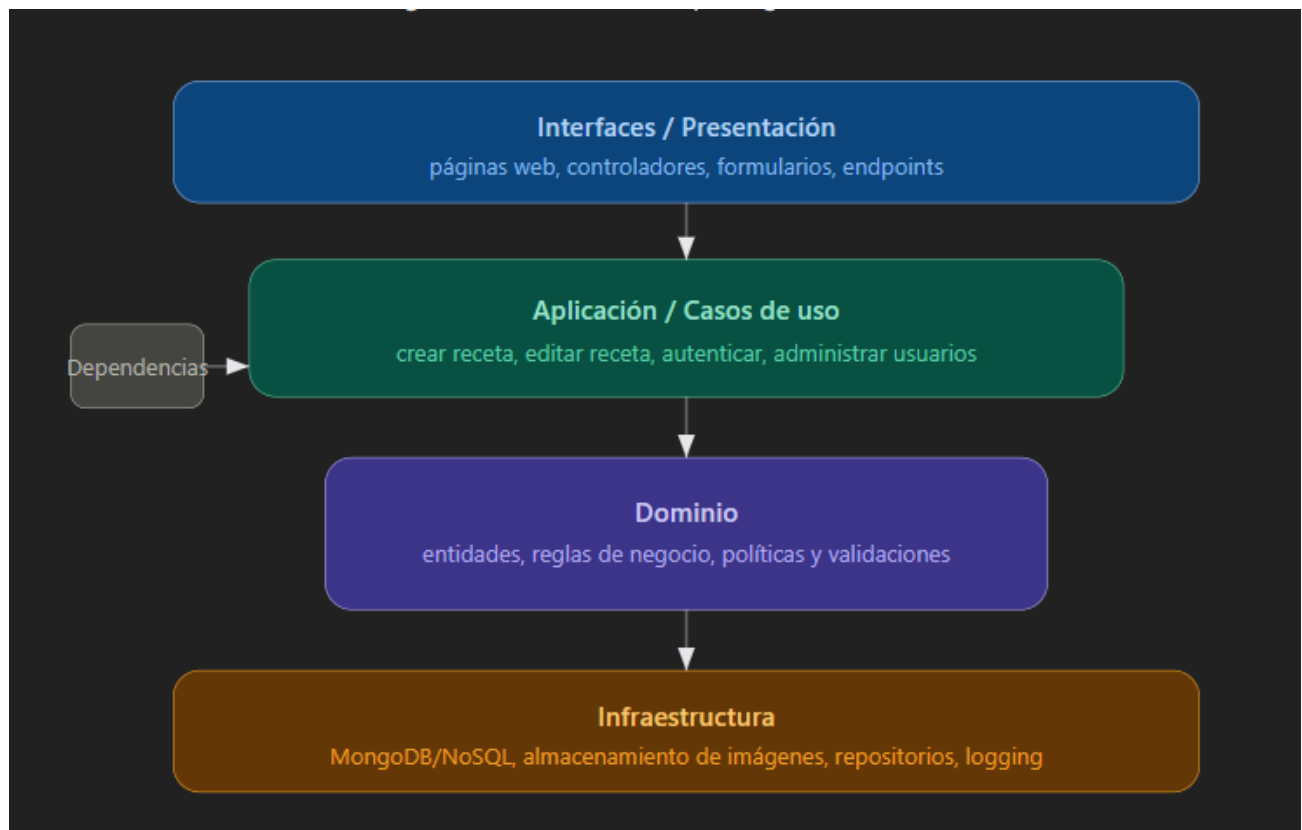
Texto descriptivo

La vista lógica separa claramente la responsabilidad de cada módulo.

El sistema evita mezclar autenticación, negocio culinario y administración, lo que facilita mantenimiento, pruebas y evolución.

3.3 Vista de desarrollo

Describe cómo se organiza el software internamente.



Capas sugeridas bajo Clean Architecture

- **Interfaces**
 - páginas web
 - controladores
 - formularios
- **Aplicación**

- casos de uso
- reglas de orquestación
- **Dominio**
 - entidades del negocio
 - reglas de validación
- **Infraestructura**
 - persistencia NoSQL
 - almacenamiento de imágenes
 - servicios externos si en el futuro los hubiera

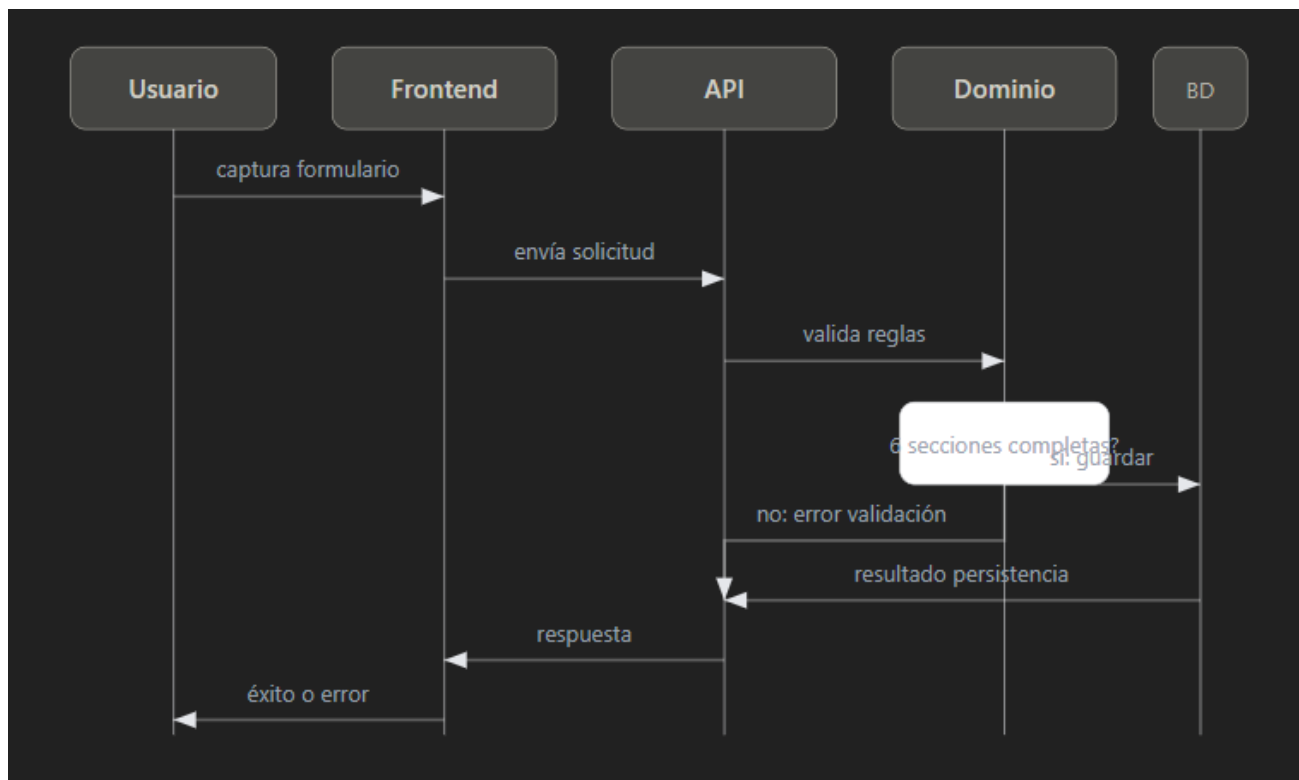
Texto descriptivo

La dependencia siempre debe apuntar hacia el dominio, nunca al revés.

Esto significa que la lógica del negocio no depende de la base de datos ni del framework web, lo que reduce acoplamiento y hace más fácil cambiar detalles técnicos sin romper reglas funcionales.

3.4 Vista de proceso

Explica el comportamiento dinámico del sistema.



Flujos relevantes

1. **Consulta pública**
 - visitante solicita listado

- sistema obtiene recetas
- sistema devuelve resultados
- 2. **Alta de receta**
 - usuario autenticado envía formulario
 - sistema valida campos obligatorios
 - sistema guarda receta
 - sistema almacena imagen
- 3. **Actualización de receta**
 - sistema verifica propietario o administrador
 - sistema aplica cambios
 - sistema registra auditoría
- 4. **Administración**
 - administrador modifica o elimina recetas o usuarios

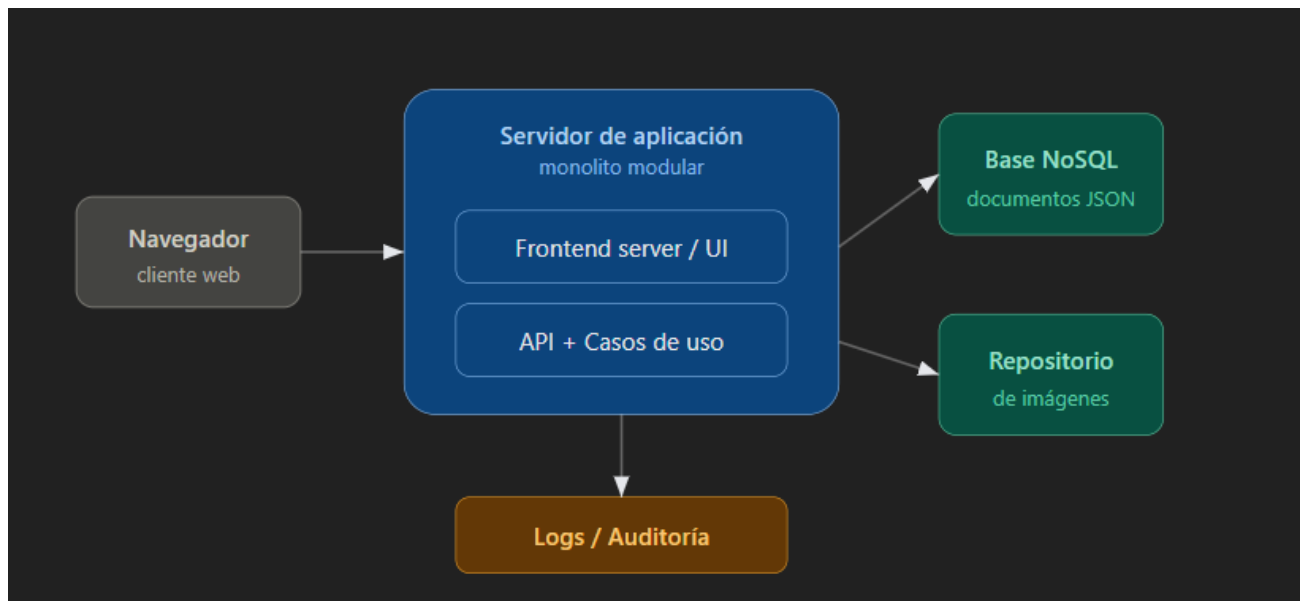
Texto descriptivo

La vista de proceso demuestra que las reglas críticas son:

- validación de información obligatoria;
- autorización basada en roles;
- restricción de edición a propietario o administrador;
- trazabilidad mínima para auditoría.

3.5 Vista física

Explica el despliegue de la solución.



Nodos principales

- Cliente web en navegador
- Servidor de aplicación web
- Base de datos NoSQL documental
- Repositorio/servicio de almacenamiento de imágenes

Texto descriptivo

La solución puede desplegarse de forma sencilla y económica:

- frontend y backend en un mismo servicio de aplicación,
- persistencia documental aparte,
- almacenamiento de imágenes desacoplado.

Esto permite un despliegue simple hoy y una evolución controlada mañana.